INSTALLATION RUNBOOK

Run SuperPlane on Azure

This document tells you how to build SuperPlane from this repository and install it on Azure Kubernetes Service (AKS).

The stack uses Azure Blob Storage, optional PostgreSQL Flexible Server, in-cluster RabbitMQ, and nginx with cert-manager.

Terraform lives in `release/terraform/aks`. The Helm chart lives in `release/superplane-helm-chart/helm`. Terraform uses that local chart by default so Azure blob, workload identity, runner API, and Fleet Manager values apply.

1. What you install

Terraform creates the Azure network, AKS, storage, identities, and the runner gallery. Helm then installs SuperPlane in the cluster.

- AKS with workload identity and Azure CNI Overlay
- NAT Gateway for outbound traffic from nodes and runner VMs
- Private Azure Blob Storage with user-delegation SAS
- Optional PostgreSQL Flexible Server, or Postgres in the cluster
- NGINX Ingress Controller with a public load balancer
- cert-manager and Let's Encrypt for TLS
- Compute Gallery definitions for Azure runner virtual machines

2. Prerequisites

- Terraform 1.5.0 or later
- Azure CLI, signed in to the target subscription
- Docker with Buildx, Helm 3, and kubectl
- Packer 1.16.1 or later if you build runner gallery images
- A DNS name that you control
- A container registry that AKS can pull (GHCR or Azure Container Registry)

3. Quota and SKU checks

New Azure subscriptions often have a 4 vCPU regional quota. The default node pool is one Standard_D2s_v4 node (2 vCPUs).

Check Flexible Server SKUs before you apply. If the list is empty, Azure will reject Flexible Server with Version should be in: []. That error is an Azure limit. It is not a SuperPlane version string.

```
az postgres flexible-server list-skus --location eastus
az vm list-skus --location eastus --size Standard_D2s_v4 --output table
```

If the SKU list is empty, set create_postgresql_flexible_server = false. Do not use Standard_D4s_v5 or Standard_D2ds_v5 unless the subscription lists those sizes.

4. Build images from this repository

Generated protobuf and OpenAPI files must exist. Run make dev.up and make pb.gen if they are missing.

```
cd /path/to/superplane
export IMAGE_TAG="local-$(git rev-parse --short HEAD)"

make image.build IMAGE=superplane IMAGE_TAG="$IMAGE_TAG"

docker build --platform linux/amd64 \
  -f release/fleet-manager/Dockerfile --target runtime \
  -t "superplane-fleet-manager:${IMAGE_TAG}" .
```

Do not run release/superplane-image/build.sh for this path. That script pushes to GHCR.

5. Push images to a registry

AKS cannot pull images that exist only on your laptop. Create Azure Container Registry, or push to GHCR if you have access.

```
export ACR_NAME="YOUR_ACR_NAME"
az group create --name superplane-images --location eastus
az acr create -g superplane-images -n "$ACR_NAME" --sku Basic -l eastus
az acr login --name "$ACR_NAME"
export ACR_LOGIN="$(az acr show -n "$ACR_NAME" --query loginServer -o tsv)"
export ACR_ID="$(az acr show -n "$ACR_NAME" --query id -o tsv)"

docker tag "superplane:${IMAGE_TAG}" "${ACR_LOGIN}/superplane:${IMAGE_TAG}"
docker tag "superplane-fleet-manager:${IMAGE_TAG}" \
  "${ACR_LOGIN}/superplane-fleet-manager:${IMAGE_TAG}"
docker push "${ACR_LOGIN}/superplane:${IMAGE_TAG}"
docker push "${ACR_LOGIN}/superplane-fleet-manager:${IMAGE_TAG}"
```

6. Configure Terraform

```
cd release/terraform/aks
cp terraform.tfvars.example terraform.tfvars
```

Set at least these values:

```
subscription_id       = "YOUR_SUBSCRIPTION_ID"
domain_name           = "superplane.example.com"
letsencrypt_email     = "admin@example.com"
location              = "eastus"
node_count            = 1
node_vm_size          = "Standard_D2s_v4"
superplane_image_tag  = "local-YOURSHA"
image_registry        = "YOURACR.azurecr.io"
container_registry_id = "/subscriptions/.../registries/YOURACR"
helm_chart_path       = "../../superplane-helm-chart/helm"
# create_postgresql_flexible_server = false
```

Terraform does not install Fleet Manager or store its installation token. Do not keep a *_override.tf file. Chart path and image registry are Terraform variables.

7. Apply

```
cd release/terraform/aks
terraform init
terraform apply
```

The first apply takes 15 to 20 minutes. Terraform assigns AcrPull when container_registry_id is set. You do not need -target for a new install.

If AKS already exists without that role, attach ACR once, then apply again:

```
az aks update --resource-group superplane --name superplane --attach-acr "$ACR_NAME"
```

8. kubectl, DNS, and first login

kubectl uses localhost:8080 until you load the AKS credentials.

```
az aks get-credentials --resource-group superplane --name superplane --admin

kubectl get svc -n ingress-nginx ingress-nginx-controller \
  -o jsonpath='{.status.loadBalancer.ingress[0].ip}'
```

Create a DNS A record for domain_name that points to that IP. Let's Encrypt HTTP-01 needs the public name. A local hosts file does not issue the certificate.

HTTPS on the load balancer IP fails until the certificate is ready. Use port-forward to open the app:

```
kubectl port-forward -n superplane svc/superplane-api 8000:8000
# Open http://localhost:8000
```

Complete owner setup. The SuperPlane chart provisions the dedicated Fleet Manager service-token Secret in Kubernetes.

```
kubectl get pods -n superplane
kubectl get certificate -n superplane
```

9. Optional Azure runner VMs

Terraform already creates the gallery, runner subnet, NSG, and identities. Packer publishes Ubuntu images into the gallery. Fleet Manager then creates one VM per runner.

```
packer init release/runner/packer/azure/runner.pkr.hcl
packer build -var-file=release/runner/packer/azure/runner.pkrvars.hcl \
  -var architecture=amd64 -var vm_size=Standard_D2ds_v4 \
  -var image_name=superplane-runner-amd64 -var image_version=1.0.0 \
  release/runner/packer/azure/runner.pkr.hcl
```

Open Set up runner fleets in SuperPlane. The wizard preloads the Terraform outputs. Replace the Azure image placeholder with the immutable gallery image version, confirm that the dedicated Secret exists, and install the standalone Fleet Manager chart.

10. Destroy

```
cd release/terraform/aks
terraform destroy
```

Flexible Server keeps backups for seven days when that server exists.

11. Troubleshooting

Symptom	Action
AKS rejects the VM size	Set node_vm_size to a size from the Azure error list. Default is Standard_D2s_v4.
ServiceCidrOverlapExistingSubnetsCidr	The stack sets aks_service_cidr to 172.16.0.0/16 so it does not overlap 10.0.0.0/16.
Insufficient regional vCPU quota	Set node_count = 1 and Standard_D2s_v4. Raise quota before you grow the pool.
kube_admin_config[0] invalid index	Terraform reads kube_admin_config_raw. Apply again after the cluster exists.
Flexible Server Version should be in: []	Set create_postgresql_flexible_server = false. Helm then runs Postgres in the cluster.
Storage 403 Key based authentication	The account keeps shared keys so Terraform can wait. Pods still use workload identity.
Helm labels unmarshal bool	Workload identity labels use type = string. The chart quotes label values.
Helm context deadline exceeded	The release waits 20 minutes. Check pod logs. Image pulls and migrate can be slow.
CrashLoopBackOff Dirty database	A killed CREATE INDEX CONCURRENTLY leaves schema_migrations dirty. Repair the index, set dirty = false, restart the API.
kubectl localhost:8080 refused	Run az aks get-credentials --resource-group superplane --name superplane --admin.

12. Azure resources this stack creates

Defaults use name prefix superplane in eastus. Objects that Azure creates for the ingress load balancer are listed as cloud-provider resources.

Resource groups

Resource	Default name	Purpose
Resource group	superplane	AKS, network, storage, identities, private DNS
Resource group	superplane-runners	Compute Gallery and runner VM identity

Networking

Resource	Default name	Purpose
Virtual network	superplane-vnet	Address space 10.0.0.0/16
Subnet	superplane-aks	AKS nodes, 10.0.0.0/20, Storage service endpoint
Subnet	superplane-postgres	Flexible Server delegation, 10.0.16.0/24
Subnet	superplane-runners	Runner VMs, 10.0.17.0/24, no inbound Internet
NAT Gateway	superplane-nat	Outbound Internet for nodes and runner VMs
Public IP	superplane-nat	NAT Gateway frontend
NSG	superplane-aks	AKS subnet
NSG	superplane-runners	Deny inbound Internet to runner VMs

AKS Overlay uses Kubernetes service CIDR 172.16.0.0/16 and kube-dns 172.16.0.10.

Kubernetes on Azure

Resource	Default name	Purpose
AKS cluster	superplane	SKU Standard. Workload identity and OIDC issuer enabled. Local admin accounts enabled.
Role assignment	Network Contributor	AKS cluster identity on the virtual network

Resource	Default name	Purpose
Role assignment	AcrPull	Kubelet identity on ACR when container_registry_id is set
Load Balancer (cloud provider)	created by ingress-nginx	Public Standard Load Balancer for HTTP and HTTPS
Public IP (cloud provider)	created with the load balancer	Ingress address. Point DNS A records here.

Data

Resource	Default name	Purpose
Storage account	sp + 8 random chars	Private blob store. TLS 1.2. GRS.
Blob container	superplane	Platform files. Private access.
Private DNS zone	privatelink.postgres.database.azure.com	Flexible Server private DNS
Private DNS VNet link	superplane-postgres	Links the zone to the VNet
PostgreSQL Flexible Server	superplane-db	Created only when create_postgresql_flexible_server is true. Default SKU B_Standard_B2s.
PostgreSQL database	superplane	Application database
PostgreSQL configuration	require_secure_transport=on	TLS required for Flexible Server

Identities and role assignments

Resource	Default name	Purpose
User-assigned identity	superplane-app	Workload identity for API, workers, and websocket
User-assigned identity	superplane-fleet-manager	Workload identity for Fleet Manager
User-assigned identity	superplane-runner	Assigned to each runner virtual machine
Federated credential	superplane-app	system:serviceaccount:superplane:superplane
Federated credential	superplane-fleet-manager	system:serviceaccount:superplane:superplane-fleet-manager
Role	Storage Blob Data Contributor	App identity on the storage account
Role	Storage Blob Delegator	Lets SuperPlane mint user-delegation SAS URLs
Role	Virtual Machine Contributor	Fleet Manager on resource group superplane-runners
Role	Network Contributor	Fleet Manager on resource group superplane-runners
Role	Reader	Fleet Manager on the Compute Gallery
Role	Managed Identity Operator	Fleet Manager on the runner identity

Runner image gallery

Resource	Default name	Purpose
Azure Compute Gallery	superplanerunners	Holds Trusted Launch image definitions
Gallery image definition	superplane-runner-amd64	linux/amd64 Ubuntu 24.04 runner base
Gallery image definition	superplane-runner-arm64	linux/arm64 Ubuntu 24.04 runner base

Packer publishes image versions into those definitions. Terraform does not build the versions.

13. What runs in the cluster (not Azure PaaS)

- cert-manager and ClusterIssuer letsencrypt-prod

- ingress-nginx controller
- SuperPlane API, workers, and websocket Deployments
- RabbitMQ StatefulSet
- Postgres StatefulSet when Flexible Server is off
- Fleet Manager, when installed separately with its standalone Helm chart

14. Created outside Terraform

- Azure Container Registry, if you build images locally
- DNS A record for the SuperPlane domain
- Packer gallery image versions
- Immutable Runner archives from the SuperPlane GitHub release
- Let's Encrypt certificate (cert-manager creates it after DNS works)

15. Source paths

Path	Role
release/terraform/aks	Azure install stack
release/superplane-helm-chart/helm	Helm chart used by this stack
release/fleet-manager-chart/helm	Standalone Fleet Manager chart
Dockerfile --target runner	SuperPlane application image
release/fleet-manager/Dockerfile	Fleet Manager image
release/runner/packer/azure	Packer gallery images
pkg/blob/azure	Azure Blob provider with workload identity
pkg/fleets/provider/azure	Fleet Manager Azure VM provider